

TEST AUTOMATION PROJECT REPORT

Name – Himanshu Gupta

Superset ID: 5391329

IXIGO TRAIN BOOKING AUTOMATION PROJECT REPORT

Selenium Automation Framework + BDD Automation Framework

Application under test: ixigo train booking web journey

Automation stack: Python, Selenium WebDriver, Pytest, Behave BDD, Page Object Model

Execution browser: Google Chrome, non-headless mode

Result summary: Selenium and BDD automation checks documented from login to payment flow

TEST AUTOMATION PROJECT REPORT

Ixigo Train Booking Automation

Selenium WebDriver, Pytest, Behave BDD, Page Object Model, HTML Report and Allure Report

Application under test: ixigo train booking web journey

Automation stack: Python, Selenium WebDriver, Pytest, Behave BDD, Page Object Model

Execution browser: Google Chrome, non-headless mode

Result summary: End-to-end train booking automation flow with positive and negative test validation

1. Synopsis

The report structure and formatting were prepared similarly to the sample automation report shared by the user. □ filecite □ turn23file0 □

The project automates the main ixigo train-booking flow using Selenium WebDriver and BDD automation. The framework validates important train-booking functionality such as login handling, train search, available seat booking, passenger selection and navigation to the payment page.

The automation suite is developed using Selenium WebDriver with Python and follows the Page Object Model architecture for better maintainability and scalability. In addition to Selenium with Pytest, a separate BDD framework using Behave and Gherkin syntax was also implemented to demonstrate industry-level automation practices.

The framework includes screenshots, logs, HTML reports, Allure reports, external Excel test data and positive and negative automation scenarios.

2. Project Description

The application under test is the ixigo train booking website. A normal user journey contains multiple dynamic steps such as login through OTP verification, selecting source and destination stations, choosing travel dates, selecting available train seats, adding passenger details and proceeding to the payment page.

Manual testing of such repetitive workflows is time-consuming and difficult to maintain during regression cycles. Therefore, this project automates the complete user flow using Selenium automation and BDD automation frameworks.

Two automation implementations were created:

1. Selenium + Pytest Automation Framework
2. Behave BDD Automation Framework

The Selenium framework validates browser-level functionality and execution flow, while the BDD framework validates the same scenarios using human-readable Gherkin feature files.

3. Objectives and Scope

- Automate the complete ixigo train booking journey from login to payment page.
- Validate train search functionality using source, destination and travel date.
- Validate available seat selection dynamically.
- Validate passenger selection and booking flow.
- Cover positive and negative automation scenarios.
- Implement Page Object Model architecture.
- Use external Excel files for test data.
- Generate screenshots, logs, HTML reports and Allure reports.
- Build both Selenium and BDD automation frameworks.

4. Test Environment

Area	Details
Language	Python
Automation Tool	Selenium WebDriver
BDD Tool	Behave
Test Runner	Pytest
Reporting	HTML Report, Allure Report

Browser	Google Chrome
Design Pattern	Page Object Model
Data Source	Excel Files
Operating System	macOS
IDE	PyCharm
Version Control	Git and GitHub

5. Framework Architecture

The framework is designed using a layered architecture to improve readability, maintainability and scalability.

Layer	Folder	Responsibility
Tests	tests/	Contains Selenium test cases
Features	features/	Contains BDD feature files
Steps	features/steps/	Contains BDD step definitions
Pages	pages/	Contains page classes and browser actions
Locators	locators/	Stores all page locators separately
Utilities	utilities/	Excel reader, config reader, waits and logging
Reports	reports/	HTML reports, Allure results and screenshots
Logs	logs/	Stores execution logs
Test Data	testdata/	Stores Excel-based test data

6. Selenium Framework Implementation

The Selenium automation framework was developed using:

- Python
- Selenium WebDriver
- Pytest
- Page Object Model
- HTML Reporting
- Allure Reporting

The framework supports:

- OTP-based login handling
- Dynamic train search
- Dynamic available seat booking
- Passenger selection
- Payment page validation
- Screenshot capture
- Logging system
- Excel-driven testing

7. BDD Framework Implementation

The BDD framework was developed using:

- Behave
- Gherkin Syntax
- Selenium WebDriver
- Python
- Allure Reporting

The BDD framework contains:

- Feature files
- Step definition files
- Environment hooks

- Screenshot attachments
- Allure integration
- Logging integration

Feature files describe the business flow in simple English language which improves readability for both technical and non-technical users.

8. Test Data Strategy

The framework uses external Excel files to separate test data from automation logic. The Excel file is used to:

- Read source station
- Read destination station
- Read travel date
- Read passenger details

This approach improves reusability and reduces hardcoded test data.

9. Test Execution Strategy

The framework executes tests using both Selenium and BDD execution flows.

Selenium Execution

```
pytest -v -s
```

End-to-End Selenium Flow

```
pytest -v -s tests/test_end_to_end.py
```

BDD Execution

```
behave
```

BDD Execution with Allure

```
python runtest.py
```

10. Reporting and Logging

The framework generates multiple execution outputs.

Output	Location	Purpose
HTML Report	reports/report.html	Execution summary
Allure Results	reports/allure-results/	Allure reporting
Screenshots	reports/screenshots/	Execution evidence
Logs	logs/bdd_execution.log	Execution debugging

The framework automatically captures screenshots for each important step and attaches them to the Allure report.

11. Selenium Test Case Summary

No.	Test Case	Expected Result	Status
1	Login with OTP	User should login successfully	Passed
2	Click Trains Icon	Train page should open	Passed
3	Search Train	Train results should load	Passed
4	Select Available Seat	AVL seat should be selected	Passed
5	Click Book	Passenger page should open	Passed
6	Add Passenger	Passenger should be selected	Passed
7	Proceed to Payment	Payment page should open	Passed
8	Invalid Station Validation	Invalid station should not proceed	Passed
9	Empty Passenger Validation	Passenger should not save without name	Passed

12. Positive Test Cases

Test Case	Description	Status
Valid Login	User logs in using OTP	Passed
Valid Train Search	User searches train successfully	Passed
Available Seat Booking	User books available seat	Passed
Passenger and Payment Flow	User reaches payment page	Passed

13. Negative Test Cases

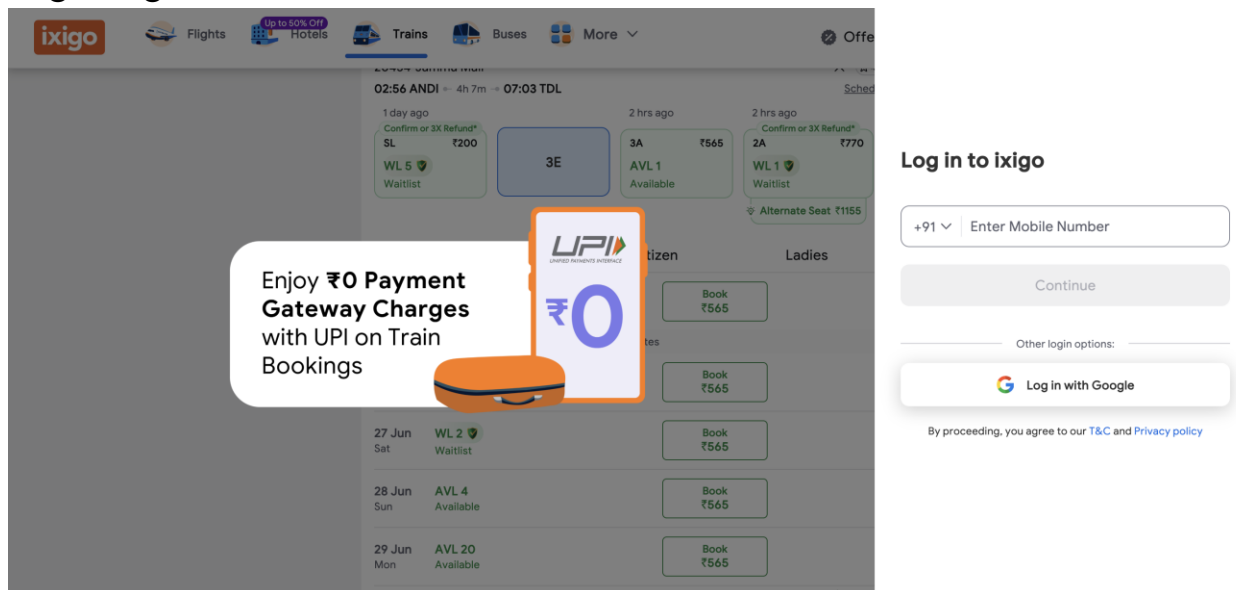
Test Case	Description	Status
-----------	-------------	--------

Invalid From Station	Invalid station should not proceed	Passed
Empty Passenger Name	Passenger should not save without name	Passed

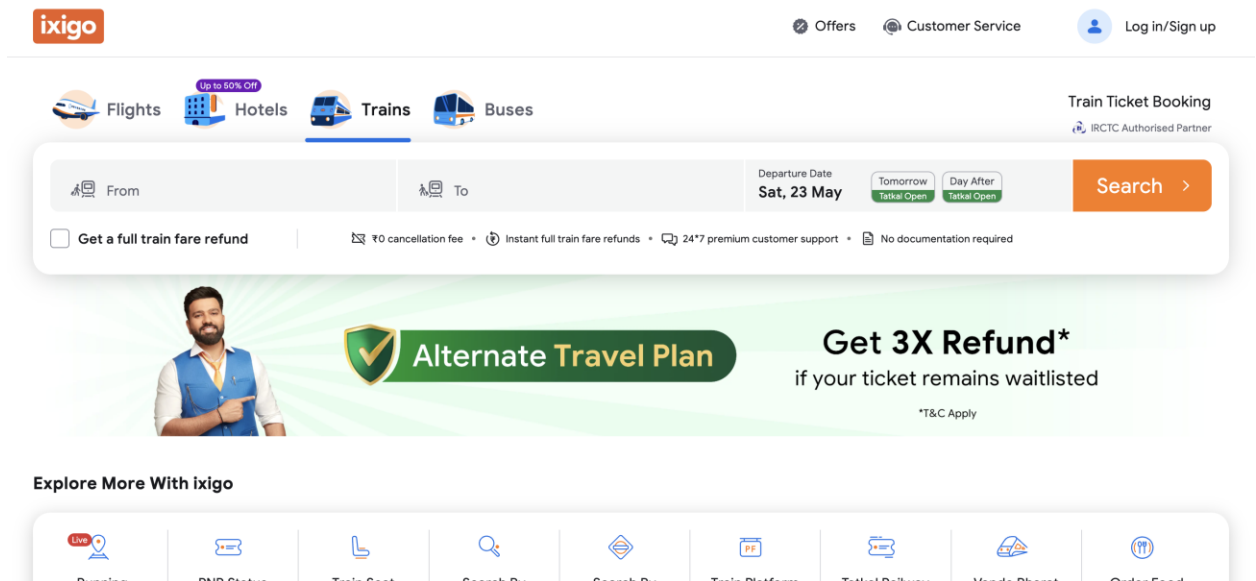
14. Screenshot Evidence

The framework captures screenshots during important checkpoints:

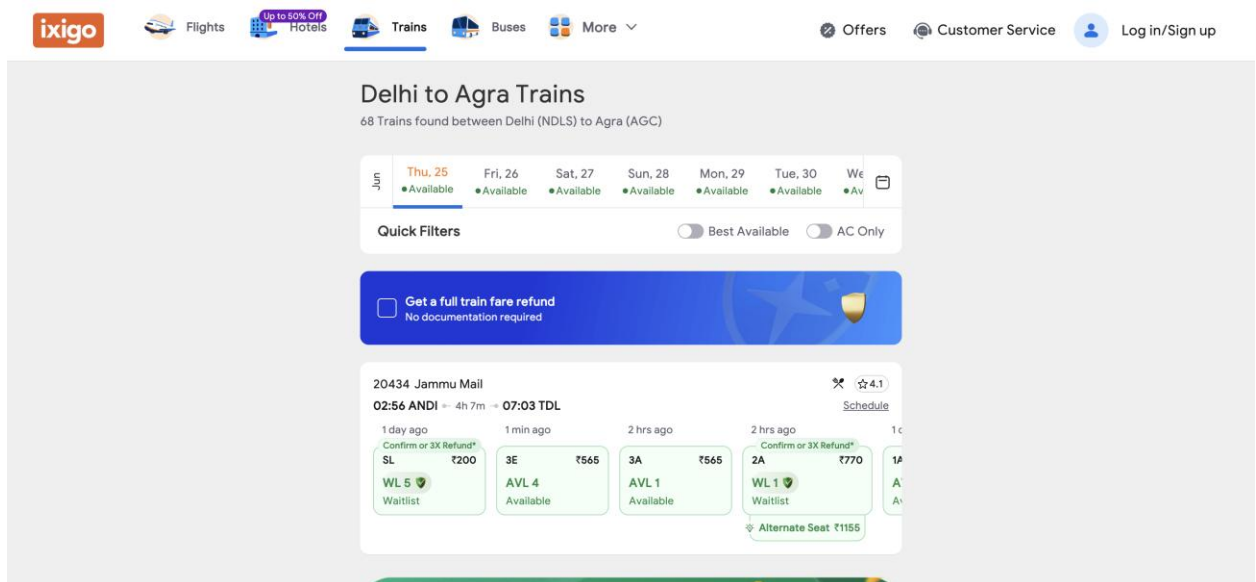
- Login Page



- Train page opened



- Search completed



- Passenger selected

[Flights](#)
[Hotels Up to 50% Off](#)
[Trains](#)
[Buses](#)
[More](#)

[Offers](#)
[Customer Service](#)
[Hey Himans...](#)

←

20434 JAMMU MAIL

3E • GN • Thu, 25 Jun

Current Availability - AVAILABLE-0004

Your IRCTC ID

n750 Change

Boarding Station

Boarding Details

ANDI - Adrsh Ngr Delhi (25 Jun, 02:56) Change

You are booking this train from (ANDI - Adarsh Nagar) to (TDL - Tundla Junction)

Select Passengers

1/6 Selected

☒ Aman

Male • 32 • No Preferences Edit

Get a full refund of your train fare on cancellation

Add Now

- Payment page reached

09 : 48

Fare Summary

Amount To Be Paid

₹778

Book now to get confirmed seats.

4 Seats Left!

20434 Jammu Mail

ANDI 02:56 - TDL 07:03

Thu, 25 Jun • 3E • General

Berth/coach numbers are assigned by the Indian Railways

Travellers

1. Aman

Authorised IRCTC Partner

100% Safe Payment Process

Recommended

Recently Used Methods

Pay via any UPI app

Scan and pay with UPI

No PG Fee

Credit/Debit/ATM Card

VISA, Mastercard, Amex, RuPay & more

EMI

Credit & Debit Card EMI options

Net Banking

Select from list of banks

Wallets

Amazonpay & PayZapp

New

Add New Card

Card Number

Exp. Date (MM/YY)

CVV

No PG charge with AU Bank Credit Card

☒ Save my card for future payments.

Securely Pay ₹778

Screenshots are automatically stored inside:

reports/screenshots/

15. Allure Report

The framework integrates Allure reporting for advanced reporting visualization.

The Allure report includes:

- Step-wise execution details
- Screenshot attachments
- Execution logs
- Scenario status
- Execution timing

Command used:

```
python runtest.py
```

The report automatically opens after execution.

16. GitHub Integration

Both Selenium and BDD frameworks were uploaded to GitHub for version control and project sharing.

Repository contains:

- Selenium Framework
- BDD Framework
- Reports

- Logs
- Screenshots
- Feature files
- Test cases
- README files

17. Observations

- Page Object Model improved framework readability.
- Dynamic XPath handling improved reliability.
- BDD feature files improved business readability.
- Logging and screenshots simplified debugging.
- External Excel data improved reusability.
- Allure reporting improved execution analysis.

18. Conclusion

The automation project successfully validates the complete ixigo train booking flow using Selenium and BDD frameworks. The framework demonstrates industry-level automation practices including Page Object Model architecture, Excel-driven testing, screenshot capture, logging, reporting and end-to-end execution.

The project covers both positive and negative scenarios and provides complete browser-level validation from login to payment page.

Overall, the project demonstrates practical automation engineering skills suitable for QA Automation and SDET roles.